



Base de données NoSQL

■ Concepts

- Volumétrie
 - ✓ Les bases de données NoSQL traitent de très grandes quantités de données plusieurs téra-octets
- Scalabilité
 - ✓ Avec explosion des taille de données à gérer, BD traditionnelles fournissent pas de temps de réponses satisfaisant, 2 solutions:
 - Augmenter les ressources de la machine (puissance du processeur, mémoire, etc) : **scalabilité verticale**
 - Utiliser des clusters de machines (en théorie augmentation illimité des capacités): **scalabilité horizontale**
 - ✓ Au-délà d'une certaine taille de données, seule la scalabilité horizontale est plausible

Base de données NoSQL - MongoDB

■ Présentation

- Développé depuis 2007 par MongoDB, travaillant sur
 - ✓ Système de Cloud Computing, Informatique à données réparties
 - ✓ Similitude au service Google App Engine
 - ✓ 1^{ère} version industrielle la 1.4, en 2010
- Base de données open source orientée documents
 - ✓ Les données sont des documents
 - Stockés en Binary JSON(BSON)
 - Documents similaires rassemblés dans des collections
 - Pas de schéma des documents définis en amont
 - Document contient (généralement) l'ensemble des informations (pas ou très peu de jointure à faire)

Base de données NoSQL - MongoDB

■ Présentation

- Point sur JSON
 - ✓ JavaScript Object Notation créé en 2005,
 - ✓ Utilise une notation appelée **littéral**
 - ✓ Format d'échange de données structurées léger
 - ✓ Schéma des données non connu (contenu dans les données)
 - ✓ Basé sur 2 notions :
 - Collection de couples clé/valeur
 - Liste de valeurs ordonnées
 - ✓ Structures possibles :
 - Objet (couple clé/valeur) :
 - { } { "nom": "louis", "age": 14 }

Base de données NoSQL - MongoDB

■ Présentation

- Point sur JSON
 - ✓ Structures possibles :
 - Tableau (collection de valeurs):
 - [] [1, 5, 23]
 - Une valeur dans objet ou tableau peut être elle-même un littéral
 - ✓ 2 types atomique (**string** et **number**) et 3 constantes (**true**, **false**, **null**)
- BSON : extension de JSON
 - ✓ Quelques types supplémentaires (identifiant spécifique, binaire, date, ...)
 - ✓ Distinction entier et réel

Base de données NoSQL - MongoDB

■ Présentation

- Notion de base de données
 - ✓ Conteneur physique pour les collections
 - ✓ Chaque base de données obtient son propre ensemble de fichiers sur le système de fichiers
 - ✓ Un seul serveur MongoDB a généralement plusieurs bases de données
- Collection
 - ✓ Groupe de documents MongoDB $\Leftrightarrow$ d'une table RDBMS
 - ✓ Elle est dans une seule base de données
 - ✓ Les documents d'une collection peuvent avoir $\neq$ champs
 - ✓ Documents d'une collection ont objectif similaire ou connexe

Base de données NoSQL - MongoDB

■ Présentation

- Document
 - ✓ Ensemble de paire clé-valeur
 - ✓ Les documents ont un schéma dynamique (les documents n'ont pas le même ensemble de champs ou structure)
 - ✓ Champs communs dans les documents d'une collection peuvent contenir différents types de données
- Relation entre terminologie SGBDR et MongoDB

Base de données NoSQL - MongoDB

■ Présentation

- Relation entre terminologie SGBDR et MongoDB

Serveur et client de base de données	
mysqld / Oracle	Mongod
mysql / sqlplus	Mongo

SGBDR	MongoDB
Base de données	Base de données
Table	Collection
Tuple / ligne	Document
colonne	Champ
Jointure de table	Documents intégrés
Clé primaire	Clé primaire (clé par défaut _id fournie par MongoDB lui-même)

Base de données NoSQL - MongoDB

■ Présentation

- Exemple de document
 - ✓ Structure de document d'un site blog qui est paire clé/valeur séparée par des virgules
 - ✓ `_id` : nombre hexadécimal assurant l'unicité de chaque document
 - On peut fournir `_id` lors de l'insertion d'un document sinon MongoDB le fait
 - `_id` est sur 12 octets :
 - 4 premiers pour l'horodatage
 - 3 suivants ID de la machine
 - 2 suivants ID du processus serveur
 - 3 restants valeur incrémentielle

```
{
  _id: ObjectId(7df78ad8902c)
  title: 'MongoDB Overview',
  description: 'MongoDB is no sql database',
  by: 'tutorials point',
  url: 'http://www.tutorialspoint.com',
  tags: ['mongodb', 'database', 'NoSQL'],
  likes: 100,
  comments: [
    {
      user: 'user1',
      message: 'My first comment',
      dateCreated: new Date(2011,1,20,2,15),
      like: 0
    },
    {
      user: 'user2',
      message: 'My second comments',
      dateCreated: new Date(2011,1,25,7,45),
      like: 5
    }
  ]
}
```


Base de données NoSQL - MongoDB

■ Installation et configuration

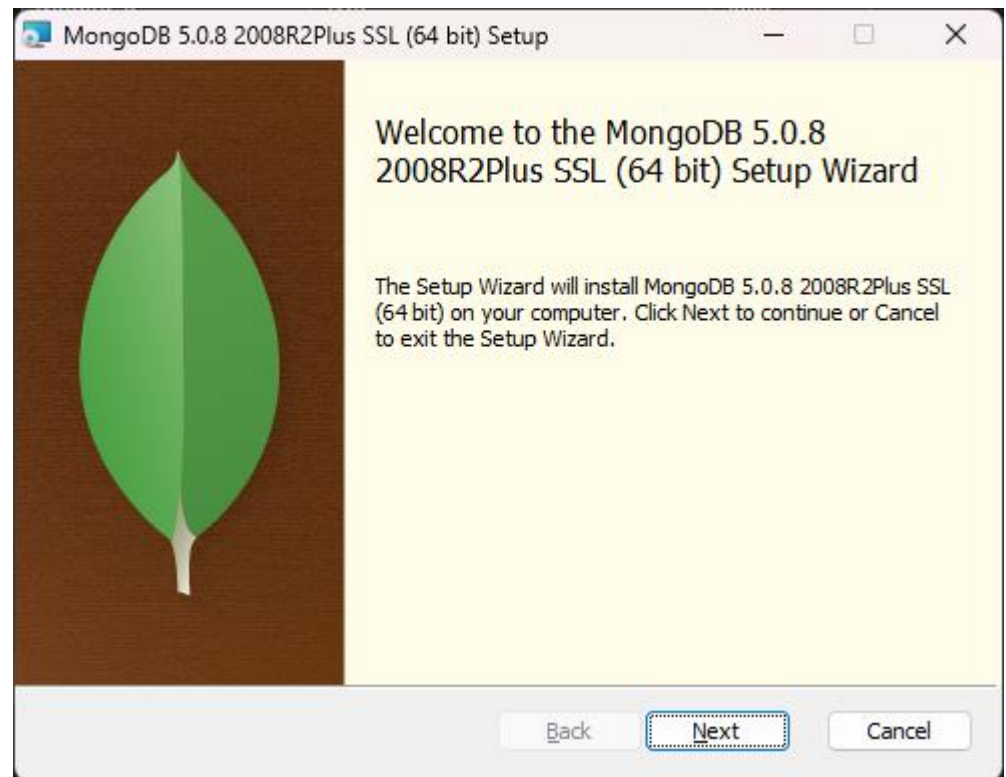
- Téléchargement de MongoDB
 - ✓ Lien : <https://www.mongodb.com/try/download/community>
- Installation
 - ✓ Sous Windows
 - Double_cliquer sur le fichier .msi
 - Et suivre les étapes
 - Définir le chemin d'accès au répertoire **bin** de **mongoDB** dans la variable d'environnement **PATH** de Windows
 - ✓ Sous Linux
 - Voir fiche de TP : installation de mongoDB sous Ubuntu

Base de données NoSQL - MongoDB

■ Installation et configuration

- Étapes d'installation sous Windows
 - ✓ Le lancement du fichier **.msi** donne:

On clique sur « **Next** »

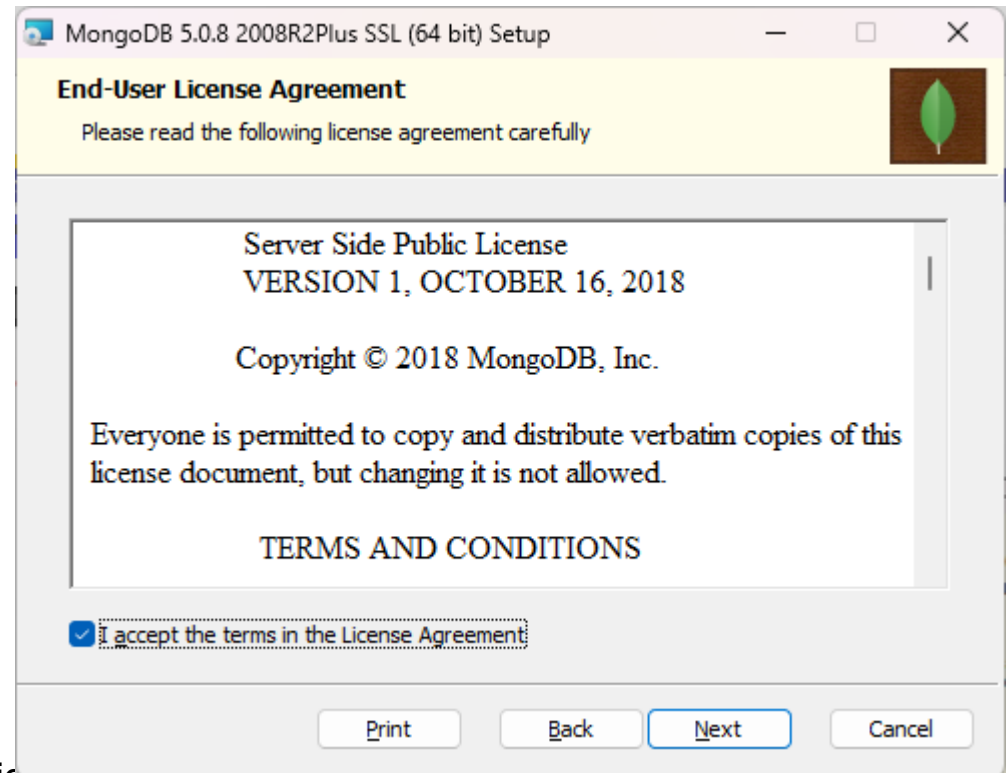


Base de données NoSQL - MongoDB

■ Installation et configuration

- Étapes d'installation sous Windows
 - ✓ On accepte les conditions :

On clique sur **Next**

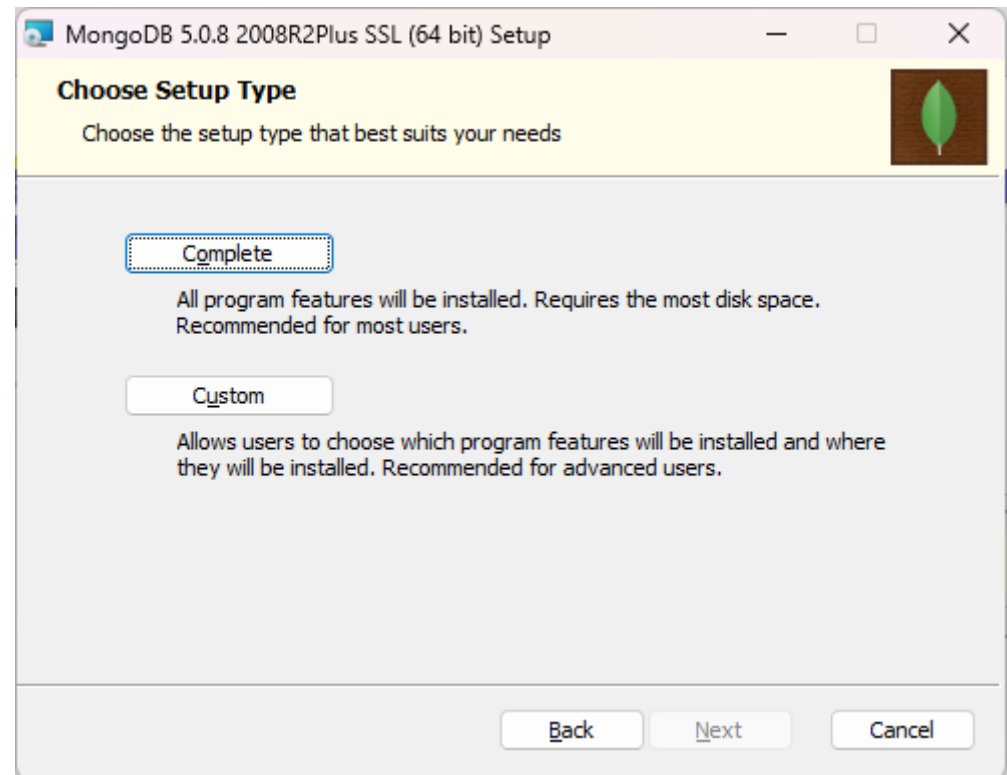


Base de données NoSQL - MongoDB

■ Installation et configuration

- Étapes d'installation sous Windows
 - ✓ On clique sur « **Custom** » pour une installation personnalisée :

En cliquant sur
« **Complete** », les
paramètres d'installation
seront définis par défaut

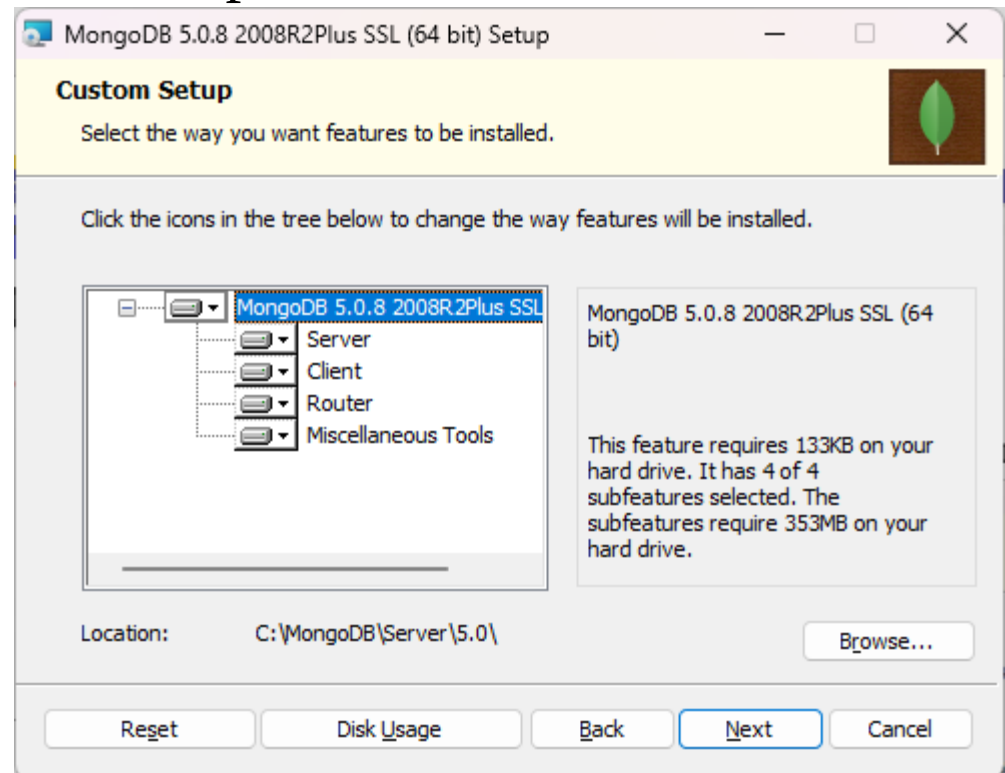


Base de données NoSQL - MongoDB

■ Installation et configuration

- Étapes d'installation sous Windows
 - ✓ À ce niveau, il est possible de spécifier:

- Les composants à installer
- Le répertoire d'installation

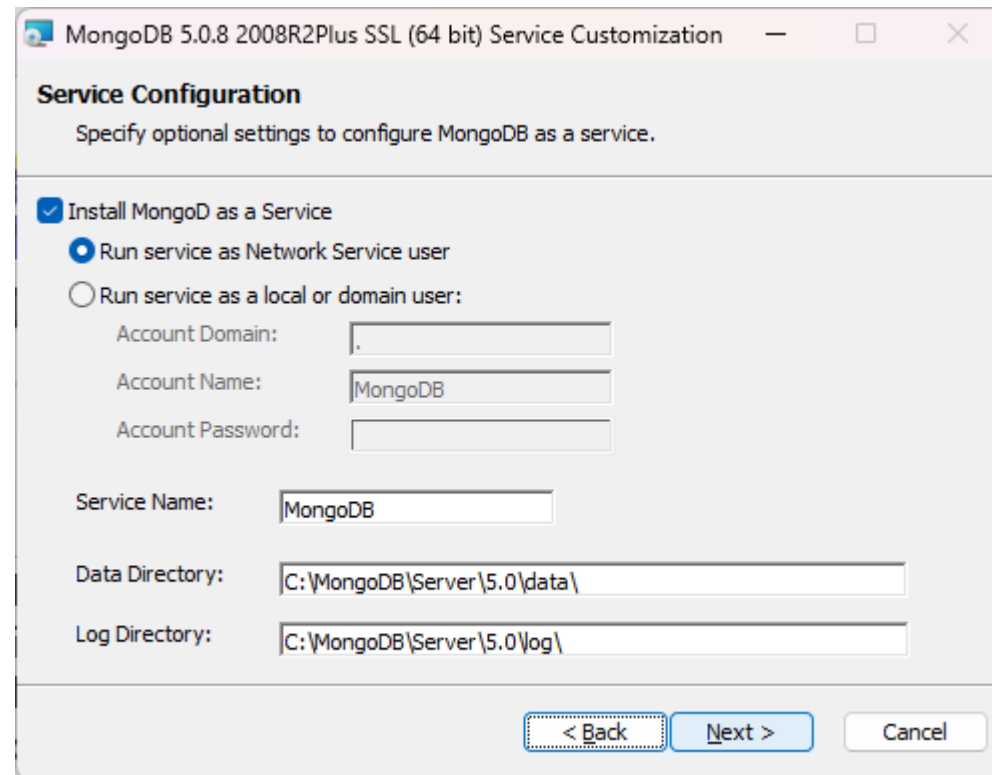


Base de données NoSQL - MongoDB

■ Installation et configuration

- Étapes d'installation sous Windows
 - ✓ On laisse l'option par défaut :

Mongo sera installé comme service réseau d'utilisateur

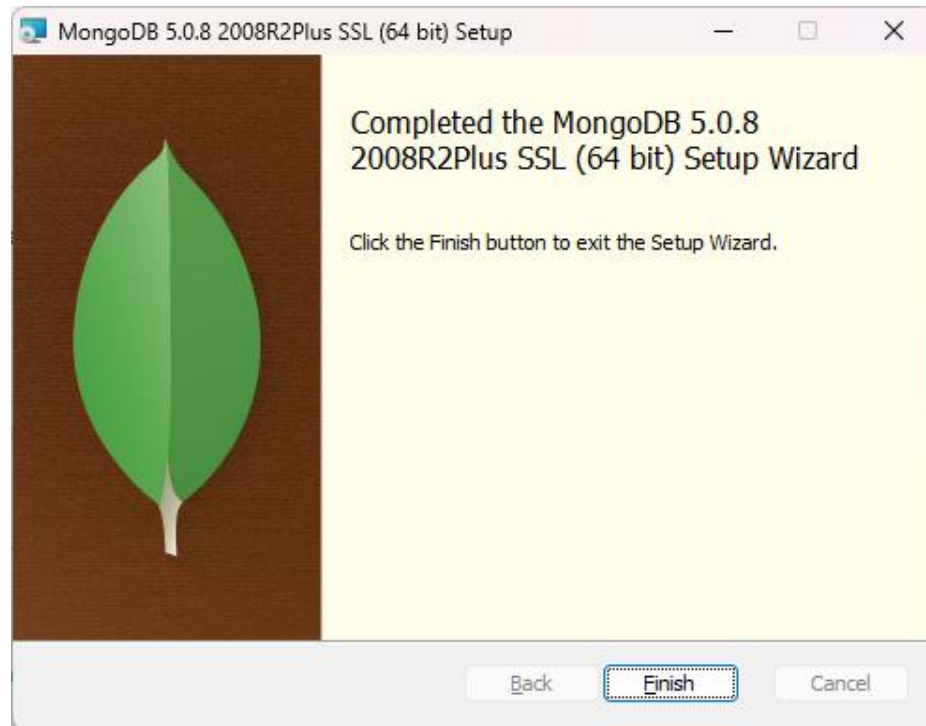
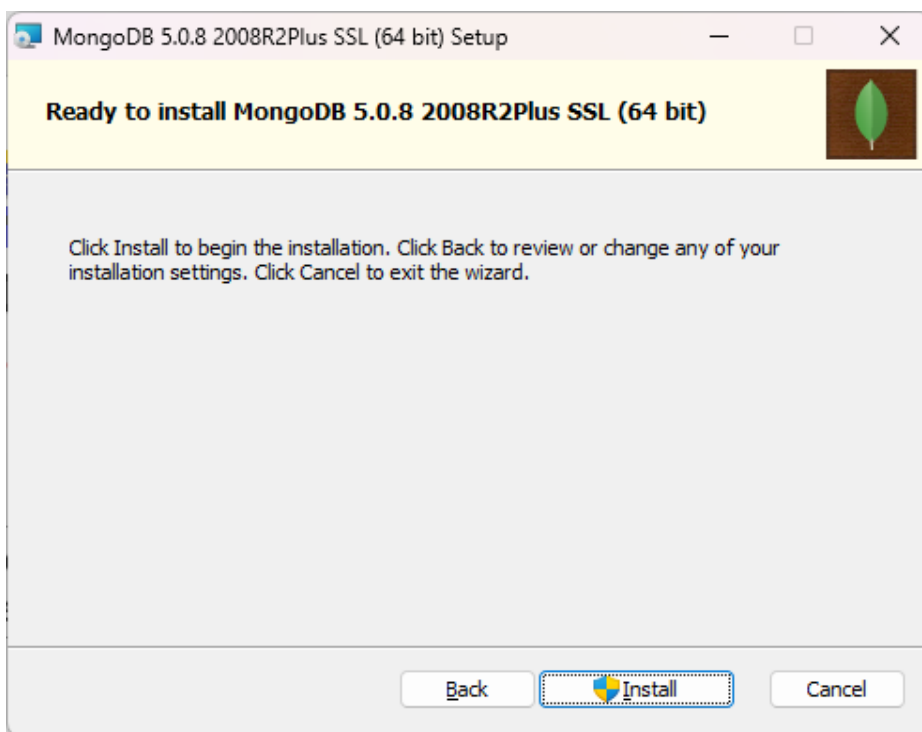


The screenshot shows the 'MongoDB 5.0.8 2008R2Plus SSL (64 bit) Service Customization' window. The 'Service Configuration' tab is active, with the instruction 'Specify optional settings to configure MongoDB as a service.' Below this, the 'Install MongoDB as a Service' checkbox is checked. Under this, the 'Run service as Network Service user' radio button is selected. The 'Run service as a local or domain user:' section is unselected. The 'Service Name' field is set to 'MongoDB'. The 'Data Directory' field is set to 'C:\MongoDB\Server\5.0\data\' and the 'Log Directory' field is set to 'C:\MongoDB\Server\5.0\log\'. At the bottom right, there are '< Back', 'Next >', and 'Cancel' buttons.

Base de données NoSQL - MongoDB

■ Installation et configuration

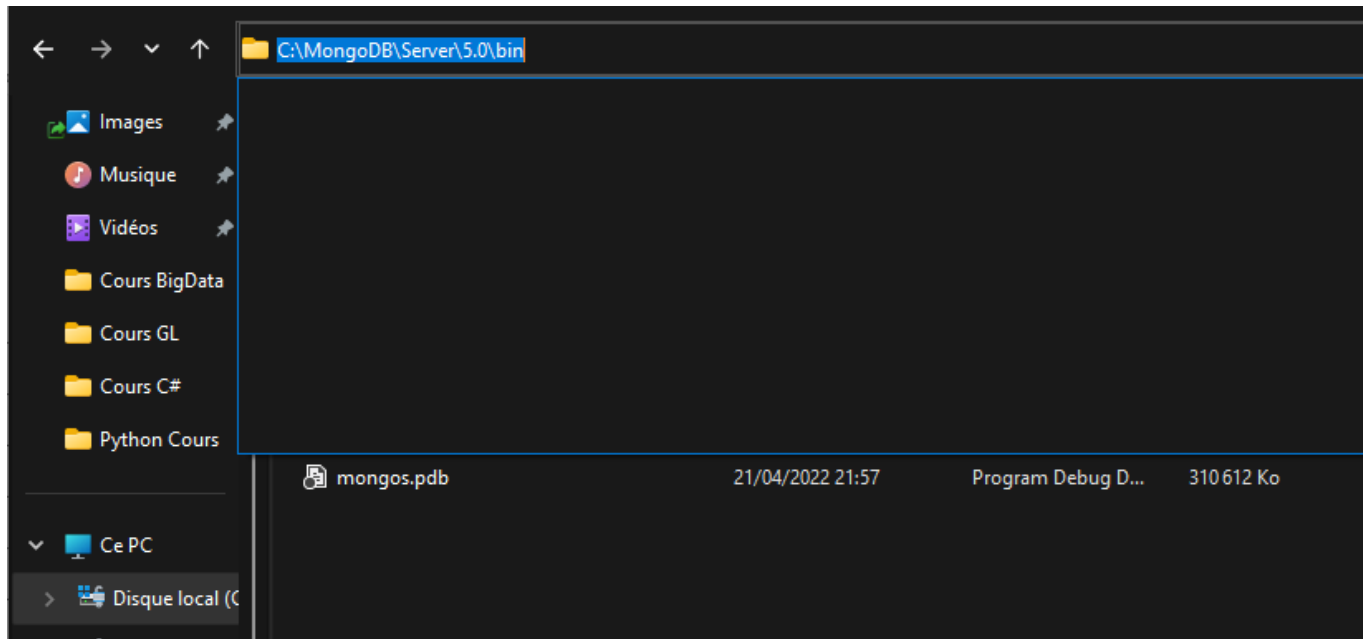
- Étapes d'installation sous Windows
 - ✓ On clique sur « **Install** », puis « **Finish** » à la fin:



Base de données NoSQL - MongoDB

■ Installation et configuration

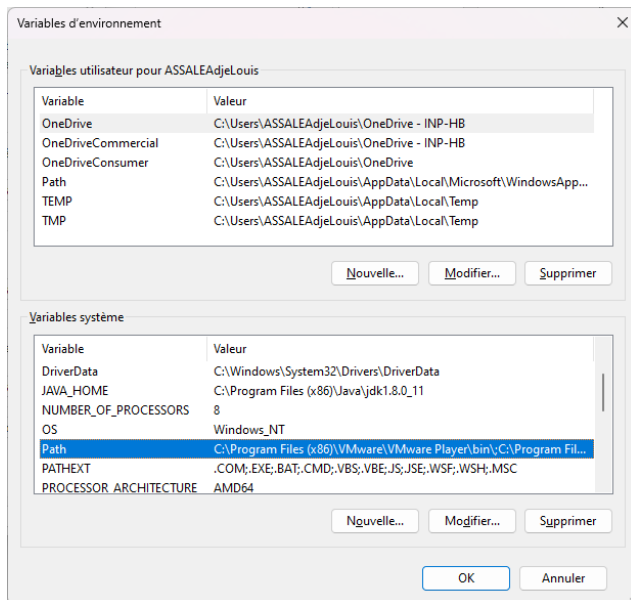
- Définir chemin d'accès au répertoire bin dans PATH
 - ✓ Dans l'explorateur Windows on accède au répertoire bin
 - Puis on fait « Ctrl + C » pour copier le chemin du répertoire bin



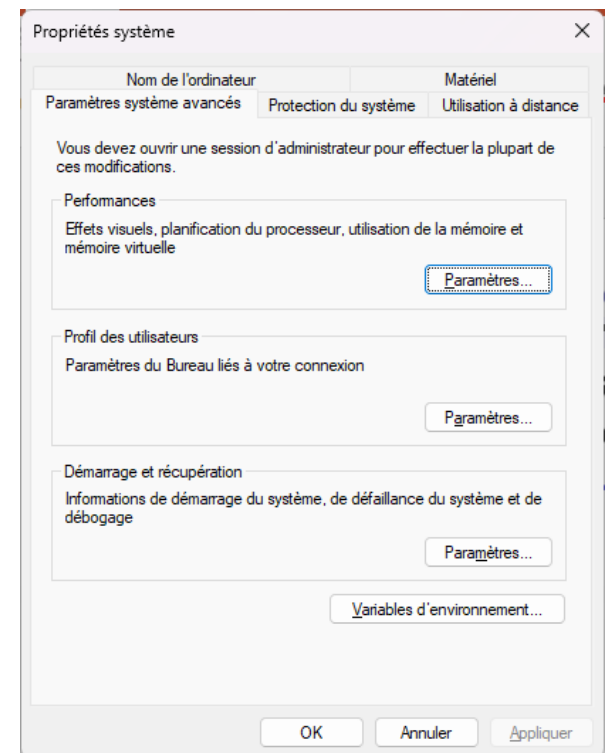
Base de données NoSQL - MongoDB

■ Installation et configuration

- Définir chemin d'accès au répertoire bin dans PATH
 - ✓ Taper « env » dans la barre de recherche
 - Cliquer sur « Modifier les variables d'environnement »
 - Choisir « Variable d'environnement »



ASSALE Adjé Louis

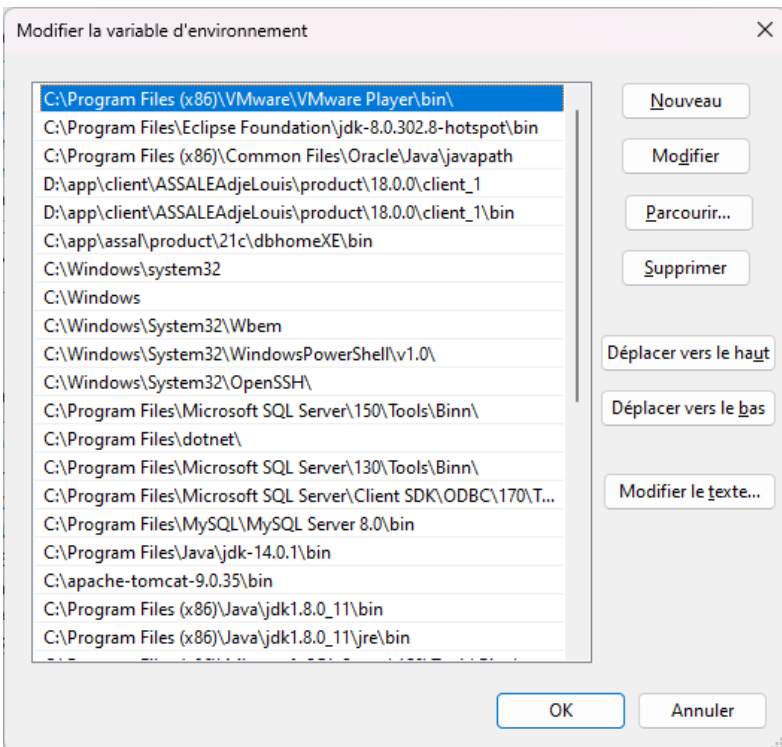


Big Data NoSQL MongoDB

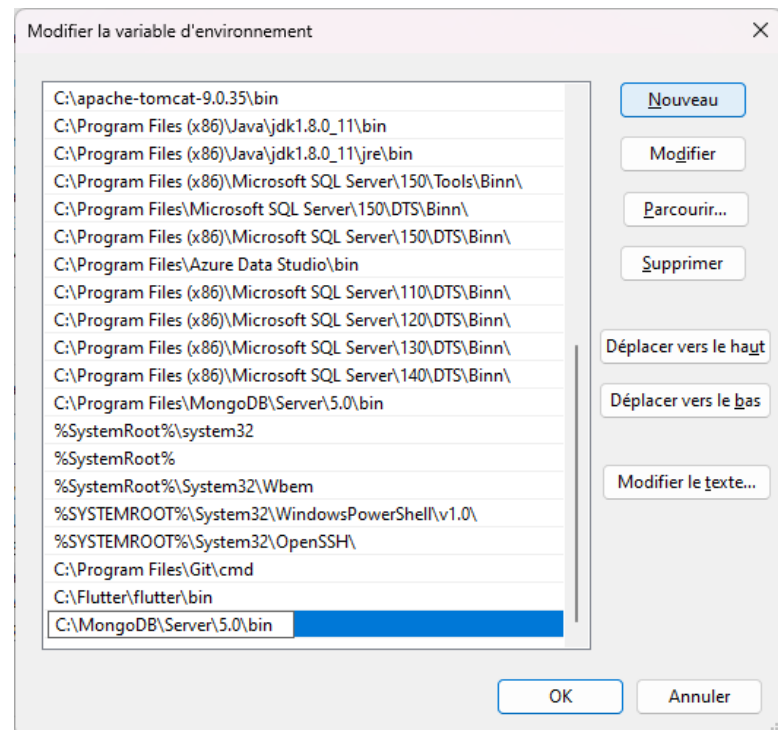
Base de données NoSQL - MongoDB

■ Installation et configuration

- Définir chemin d'accès au répertoire bin dans PATH
 - ✓ Double-cliquer sur Path dans « variables système », puis « Nouveau » et « Ctrl + V » pour coller le chemin



Big Data



Base de données NoSQL - MongoDB

■ Installation et configuration

- Répertoire de stockage de données de MongoDB
 - ✓ Par défaut le chemin relatif (à partir du répertoire d'installation) de ce répertoire est **\data\db** (à créer)
 - ✓ Possibilité de spécifier un autre chemin en utilisant l'option **--dbpath** avec mongod.exe ;
 - `C:\mongodb\bin\mongod.exe --dbpath "C:\test\mongo\data"`
 - Lancement de MongoDB
 - ✓ On exécute le fichier mongod.exe
 - `C:\mongodb\bin\mongod.exe`
 - Connexion à mongoDB
 - ✓ On exécute le fichier **mongo.exe**
- Port par défaut 27017 sur l'interface locale
 - Pour se connecter à un serveur sur port et interface ≠ te, utiliser paramètres **--port** et **--host**

Base de données NoSQL - MongoDB

■ Installation et configuration

- Outil **mongosh.exe**
 - ✓ L'outil **mongo.exe** est déprécié, **mongosh.exe** est préféré
 - ✓ Télécharger : <https://www.mongodb.com/try/download/shell>
 - ✓ Dézipper l'archive dans le dossier d'installation de mongodb
 - ✓ Mettre le chemin du dossier **/bin** de mongosh dans le **PATH**
 - ✓ Dorénavant, pour lancer mongodb, faire à l'invite de ligne de commande:
 - **C:\.....>mongosh.exe**

Base de données NoSQL - MongoDB

■ Création d'objets

- Création d'une base de données
 - ✓ Commande : `use <nom_base_de_données>`
 - Crée une nouvelle base de données si elle n'existe pas, sinon MongoDB utilise la base de données spécifiée (`test` base de données par défaut)
 - Exemple : `> use mabase` crée une nouvelle base de données
`switched to db mabase` « mabase » et la sélectionne
 - ✓ Commandes `db` et `show dbs` :
 - Afficher la base de données courante : `db`
 - Afficher la liste des bases de données (≧ au moins 1 document) : `show dbs`
 - Afficher l'aide : `help`
 - ✓ méthode : `db.dropDatabase()` : supprime la base de données en cours d'utilisation

Base de données NoSQL - MongoDB

■ Création d'objets

- Création d'une collection
 - ✓ méthode : `db.createCollection(nom, options)`
 - nom : nom de la collection
 - options (facultatif) : document utilisé pour spécifier la configuration de la collection
 - Exemple 1 : sans options
 - `>db.createCollection("maCollection")`
`{"ok" : 1}`
`>`

Base de données NoSQL - MongoDB

■ Création d'objets

- Création d'une collection
 - ✓ On peut créer automatiquement une collection en insérant un document
 - ✓ commande : `show collections` affiche les collections
 - Exemple :
 - `> show collections`
`macollection`
`>`
 - ✓ Suppression collection méthode: `db.<nom_collection>.drop()`
 - Exemple :
 - `>db.macollection.drop()`
`true`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Types de données MongoDB
 - ✓ **String** : Type de données couramment utilisé : UTF-8 valide
 - ✓ **Integer** : stocker valeur numérique, 32 ou 64 bits selon serveur
 - ✓ **Boolean** : true ou false
 - ✓ **Double** : valeur à virgule flottante
 - ✓ **Min/Max keys** : utilisé pour comparer valeur aux élément BSON les plus bas et les plus élevés
 - ✓ **Arrays** : stocker tableaux, listes ou plusieurs valeurs dans une seule clé
 - ✓ **Timestamp** : pour l'horodatage (document modifié ou ajouté)

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Types de données MongoDB
 - ✓ **Object** : utilisé pour document incorporé
 - ✓ **Null** : utilisé pour stocker valeur nulle
 - ✓ **Symbol** : comme les chaînes mais réservé aux langues
 - ✓ **Date** : stocker date et heure à la manière d'Unix
 - ✓ **Object ID** : pour stocker l'ID du document
 - ✓ **Binary data** : stocker des données binaires
 - ✓ **Code** : pour stocker le code JavaScript dans le document
 - ✓ **Regular expression** : ce type de données est utilisé pour stocker des expressions régulières.

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Insertion de documents
 - ✓ Méthode : `insert()` ou `save()`
 - Syntaxe : `db.<nom_collection>.insert(document)`
 - Si la collection `nom_collection` n'existe pas, elle est créée
 - Exemple 1: créons 2 documents simple j et k et insérons les dans la collection `macollection`
 - ```
> j = { name = "mongo" }
 { "name" = "mongo" }
```
      - ```
> k = { x = 3 }  
  { "x" = 3 }
```
 - ```
> db.macollection.insert(j)
WriteResult({ "nInserted" : 1 })
```
      - ```
> db.macollection.insert(k)  
WriteResult({ "nInserted" : 1 })
```

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Insertion de documents

- Exemple 2 :

- ```
> db.mcollection.insert({
 _id : ObjectId("507f191e810c19729de860ea"),
 title: "MongoDB Overview",
 description: "MongoDB is no sql database",
 by: "tutorials point",
 url: "http://www.tutorialspoint.com",
 tags: ['mongodb', 'database', 'NoSQL'],
 likes: 100
})
WriteResult({ "nInserted" : 1 })
```
    - ```
>
```

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Insertion de documents
 - Exemple 3 : on peut passer un tableau de documents
 - ```
> db.mcollection.insert([
 {
 title: "MongoDB Overview",
 description: "MongoDB is no sql database",
 by: "tutorials point"
 },
 {
 title: "noSQL database",
 tags: ['mongodb', 'database', 'NoSQL'],
 likes: 100
 }
])
BulkWriteResult({ "nInserted" : 1 })
```

# Base de données NoSQL - MongoDB

## ■ Utilisation de MongoDB

- Insertion de documents
  - Exemple 3 : on peut passer un tableau de documents
    - `> db.mcollection.insert([ ...`

```
]])
 BulkWriteResult({
 "writeErrors" : [],
 "writeConcernErrors" : [],
 "nInserted" : 2,
 "nUpserted" : 0,
 "nMatched" : 0,
 "nModified" : 0,
 "nRemoved" : 0,
 "upserted" : []
 })
```

# Base de données NoSQL - MongoDB

## ■ Utilisation de MongoDB

- Insertion de documents
  - ✓ Tout document mongoDB doit avoir un champ « `_id` » avec une valeur unique
    - S'il n'est pas spécifié mongo crée une valeur « ObjectId » unique pour le champ `_id`
  - ✓ Méthodes `insertOne()` et `insertMany()`
    - `insertOne()` : on insère un seul document passé en paramètre
    - `insertMany()` : on insère un tableau de documents passé en paramètre
  - ✓ Insertion de +sieurs documents en utilisant boucle « for »
    - ```
>for (var i = 1; i <= 10; i++) {  
    db.testdata.insert ( { x : i } )  
}
```

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Insertion de documents
 - ✓ L'on peut exécuter un script JavaScript (fichier contenant des commandes MongoDB) à l'aide de l'instruction **load()**
 - Syntaxe : `load(fichier)`
 - fichier : nom du fichier js précédé du chemin d'accès
 - Le chemin est relatif par défaut
 - Exple: soit le fichier cinema.js du répertoire d:\donnees_TP contenant des commandes d'insertion de données dans la collection cinema
 - `>load("D:\Donnees_TP\cinema.js")`
 - Cette commande va exécuter toutes les commandes du fichier cinema.js

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Insertion de documents
 - ✓ Manipulation de date, les fonctions : `Date()`, `new Date(yyy)`, ...
 - Fonction : `Date()`
 - Appelée simplement renvoie la date courante
 - `>db.mycol.insert({titre:"ah les femmes", auteur:"biton coulivaly", date_achat:Date()})`
`WriteResult({ "nInserted" : 1 })`
 - `>db.mycol.find().pretty()`
`{ "_id" : ObjectId("6381d45998113fb3d8ab91a1"),`
 `"titre" : "ah les femmes",`
 `"auteur" : "biton coulivaly",`
 `"date_achat" : "Sat Nov 26 2022 08:54:49 GMT+0000 (Greenwich)"`
`}`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Insertion de documents
 - ✓ Manipulation de date, les fonctions : `Date()`, `new Date(yyy)`
 - Fonctions : `new Date(yyy)`, `new ISODate(yyy)` ou `ISODate(yyy)`
 - Elles sont toutes identiques car font appel au wrapper `ISODate`
 - `yyy` : spécifie la chaîne de la date et heure à la milliseconde près
 - `>db.mycol.insert({titre:"ah les hommes", auteur:"biton coulibaly", date_achat: new Date("2015-11-28")})`

```
WriteResult({ "nInserted" : 1 })
```

```
■ >db.mycol.find({titre:"ah les hommes"}).pretty()
{  "_id" : ObjectId("6381d92d98113fb3d8ab91a2"),
    "titre" : "ah les hommes",
    "auteur" : "biton coulibaly",
    "date_achat" : ISODate("2015-11-28T00:00:00Z")}
```

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Insertion de documents
 - ✓ Manipulation de date, les fonctions : `Date()`, `new Date(xxx)`
 - Fonctions : `new Date(xxx)`, `new ISODate(xxx)` ou `ISODate(xxx)`
 - `>db.mycol.insert({titre:"Tribaliques", auteur:"Henry lopez", date_achat: ISODate("2015-11-28T14:30:00.000Z")})`
`WriteResult({ "nInserted" : 1 })`
 - `>db.mycol.find({titre:"Tribaliques"}).pretty()`
{
 "_id" : ObjectId("6381dbad98113fb3d8ab91a3"),
 "titre" : "Tribaliques",
 "auteur" : "Henry lopez",
 "date_achat" : ISODate("2015-11-28T14:30:00Z")
}

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents

- ✓ Méthode : `find()`

- Affiche tous les documents de manière non structurée

- Exemple : `>db.mcollection.find()`

- `{ "_id" : ObjectId("62af6bdb7578f5ecceba161"), "name" : "mongo" }`

- `{ "_id" : ObjectId("62af6d807578f5ecceba162"), "x" : 3 }`

- `{ "_id" : ObjectId("507f191e810c19729de860ea"), "title" : "MongoDB Overview", "description" : "MongoDB is no sql database", "by" : "tutorials point", "url" : "http://www.tutorialspoint.com", "tags" : [ "mongodb", "database", "NoSQL" ], "likes" : 100 }`

- `{ "_id" : ObjectId("62af7ceb7578f5ecceba163"), "title" : "MongoDB Overview", "description" : "MongoDB is no sql database", "by" : "tutorials point" }`

- `{ "_id" : ObjectId("62af7ceb7578f5ecceba164"), "title" : "noSQL database", "tags" : [ "mongodb", "database", "NoSQL" ], "likes" : 100 }`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents

- ✓ Méthode : `find().pretty()`

- Formate l'affichage des documents

- Exemple : `>db.mcollection.find().pretty()`

```
{ "_id" : ObjectId("62af6bdb7578f5eccebaf161"), "name" : "mongo" }
{ "_id" : ObjectId("62af6d807578f5eccebaf162"), "x" : 3 }
{
  "_id" : ObjectId("507f191e810c19729de860ea"),
  "title" : "MongoDB Overview",
  "description" : "MongoDB is no sql database",
  "by" : "tutorials point",
  "url" : "http://www.tutorialspoint.com",
  "tags" : [
    "mongodb",
    "database",
```

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents
 - ✓ Méthode `find()` peut prendre des arguments :
 - Le 1^{er} pour spécifier la sélection : (documents vérifiant tel critère)
 - Le 2nd pour spécifier la projection : (les champs à afficher)
 - On définit une liste de champs avec la valeur 1 ou 0
 - 1 pour afficher le champ – 0 pour ne pas l'afficher
 - champ `_id` s'affiche (défaut), spécifier `_id:0` pour ne pas l'afficher
 - Exemple : afficher uniquement le titre des documents de la catégorie IA
 - `>db.lacollection.find({"categorie":"IA"}, {"titre":1, "_id":0})`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents
 - ✓ Méthode `find()` peut prendre des arguments :
 - Les documents peuvent contenir des documents imbriqués. Dans ce cas là, on peut utiliser la notation pointée « `nomDocument.NomChamp` »
 - Afficher les films ayant gagné 7 « awards »
 - `>db.films.find({"awards.wins":7},{title:1,year:1,awards:1,_id:0})`

```
{
  "title": "Shunter Island",
  "year": 2010,
  "awards": {
    "wins":7,
    "nominations":53,
    "text":"7 win & 53 nomination"
  }
},
{
  "title": "Iron Man 2",
  "year": 2010,
  "awards": {
    "wins":7,
    "nominations":40,
    "text":"nominated for 1 Oscar, another 7 wins &
40 nomination"
  }
}
```

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents
 - ✓ Méthode : `findOne()`
 - Renvoie un seul document formaté
 - Exemple : `>db.mcollection.findOne({title:"MongoDB Overview"})`

```
{
  "_id" : ObjectId("507f191e810c19729de860ea"),
  "title" : "MongoDB Overview",
  "description" : "MongoDB is no sql database",
  "by" : "tutorials point",
  "url" : "http://www.tutorialspoint.com",
  "tags" : [
    "mongodb",
    "database",
    "NoSQL«
  ], ...
```

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents

- ✓ Méthodes : `limit()`

- accepte un argument numérique pour limiter le nombre de documents

- Syntaxe : `db.<nom_collection>.find().limit(nbre)`

- Exemple : soit la collection mycol suivante :

```
{_id : ObjectId("507f191e810c19729de860e1"), title: "MongoDB Overview"},  
{_id : ObjectId("507f191e810c19729de860e2"), title: "NoSQL Overview"},  
{_id : ObjectId("507f191e810c19729de860e3"), title: "Tutorials Point Overview"}
```

- `>db.mycol.find({},{"title":1,_id:0}).limit(2)` affichera :

```
{"title":"MongoDB Overview"}
```

```
{"title":"NoSQL Overview"}
```

- ✓ Méthode : `count()` renvoie le nombre de documents du résultat

- `>db.mycol.find({}).count()`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents

- ✓ Méthode : `skip()`

- accepte un argument numérique pour ignorer un nombre de documents
- Syntaxe : `db.<nom_collection>.find().limit(nbre).skip(nbre)`
- Exemple : on reprend la collection précédente
- `>db.mycol.find({},{"title":1,_id:0}).limit(2).skip(1)` affichera :
`{"title":"NoSQL Overview"}`

- ✓ Méthode : `sort({key:1|-1})`

- Trie les résultats selon le champ key – 1 pour l'ordre croissant
- `>db.mycol.find({},{"title":1,_id:0}).sort("title":-1)`
`{_id : ObjectId("507f191e810c19729de860e3"), title: "Tutorials Point Overview"}`
`{_id : ObjectId("507f191e810c19729de860e2"), title: "NoSQL Overview"},`
`{_id : ObjectId("507f191e810c19729de860e1"), title: "MongoDB Overview"}`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB : opérateurs

Opération	Syntaxe	Exemple	Équivalent du SGBDR
Égalité	{<key>: {\$ eg: <valeur>}}	db.mycol.find ({"by": "tutorials point"}). pretty ()	où by = 'tutorials point'
Moins que	{<key>: {\$ lt: <valeur>}}	db.mycol.find ({"aime": {\$ lt: 50}}). pretty ()	où aime < 50
Inférieur à égal	{<key>: {\$ lte: <valeur>}}	db.mycol.find ({"aime": {\$ lte: 50}}). pretty ()	où aime <= 50
Plus grand que	{<key>: {\$ gt: <valeur>}}	db.mycol.find ({"aime": {\$ gt: 50}}). pretty ()	où aime > 50
Supérieur à égal	{<key>: {\$ gte: <value>}}	db.mycol.find ({"aime": {\$ gte: 50}}). pretty ()	où aime >= 50
Pas égal	{<key>: {\$ ne: <valeur>}}	db.mycol.find ({"aime": {\$ ne: 50}}). pretty ()	où aime! = 50
Valeurs dans un tableau	{<key>: {\$ in: [<value1>, <value2>, <valueN>]}}	db.mycol.find ({"nom": {\$ in: ["Raj", "Ram", "Raghu"]}}). pretty ()	Où le nom correspond à l'une des valeurs de: ["Raj", "Ram", "Raghu"]
Les valeurs ne sont pas dans un tableau	{<key>: {\$ nin: <valeur>}}	db.mycol.find ({"nom": {\$ nin: ["Ramu", "Raghav"]}}). pretty ()	Où les valeurs de nom ne sont pas dans le tableau: ["Ramu", "Raghav"] ou, n'existe pas du tout

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents

✓ Et dans mongoDB : *\$and*

- Syntaxe :

```
db.nom_collection.find({$and:[{<key1>:<valeur1>},{<key2>:<valeur2>}]}).pretty()
```

- Exemple :

```
>db.mcollection.find({$and:[{title:"MongoDB Overview"},{by:"tutorials point"}]}).pretty()
{
```

```
  "_id" : ObjectId("507f191e810c19729de860ea"),
  "title" : "MongoDB Overview",
  "description" : "MongoDB is no sql database",
  "by" : "tutorials point",
  "url" : "http://www.tutorialspoint.com", ...
```

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents

- ✓ Ou dans mongoDB : \$or

- Syntaxe :

- ```
db.nom_collection.find({$or:[{<key1>:<valeur1>},{<key2>:<valeur2>}]}).pretty()
```

- Exemple :

- ```
>db.testdata.find({$or:[{x:3},{x: 6}]}).pretty()
```

- ```
{ "_id" : ObjectId("62af84157578f5ecceba167"), "x" : 3 }
```

- ```
{ "_id" : ObjectId("62af84157578f5ecceba16a"), "x" : 6 }
```

- ✓ And et Or ensemble dans mongoDB

- 2 conditions séparées par une virgule, l'une contenant un \$or

- Expl :

```
>db.testdata.find({x:{$gt:8},$or:[{x:8},{x:9}]})
```

- ```
{ "_id" : ObjectId("62af84157578f5ecceba16d"), "x" : 9 }
```

# Base de données NoSQL - MongoDB

## ■ Utilisation de MongoDB

- Recherche de documents

- ✓ Pas dans mongoDB : **\$not**

- Syntaxe :

- ```
db.nom_collection.find({$not:[{<key1>:<valeur1>},{<key2>:<valeur2>}]}).pretty()
```

- Exemple :

- ```
>db.testdata.find({x: {$not:{$gt :3}}}).pretty()
```

- ```
{ "_id" : ObjectId("62af84147578f5ecceba165"), "x" : 1 }
```

- ```
{ "_id" : ObjectId("62af84157578f5ecceba166"), "x" : 2 }
```

- ```
{ "_id" : ObjectId("62af84157578f5ecceba167"), "x" : 3 }
```

- ✓ Ni dans mongoDB : **\$nor**

- Expl :

```
>db.testdata.find({$nor:[{x:8},{x:9}]})
```

- affiche tout sauf 8 et 9

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents

- ✓ Opérateurs de recherche dans tableau : **\$in** et **\$all**

- Exple: afficher les films avec Leonardo Dicaprio **ou** Tom Hanks

- `>db.films.find({"actors":{"$in":["Leonardo DiCaprio", "Tom Hanks"]}}, {"title":1,"year":1,"actors":1,_id:0}).pretty()`

```
{
  "title": "Shunter Island"
  "year": 2010
  "actors": [
    "Leonardo DiCaprio",
    "Mark Ruffalo",
    "Ben Kingsley",
    "Max von Sydow"
  ]
}

{
  "title": "Toy Story 3"
  "year": 2010
  "actors": [
    "Tom Hanks",
    "Tim Allen",
    "Joan Cusack"
  ]
}

{
  "title": "Saving Mr. Banks"
  "year": 2013
  "actors": [
    "Emma Thompson",
    "Tom Hanks"
  ]
}
```

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents
 - ✓ Opérateurs de recherche dans tableau : `$in` et `$all`
 - Exple : films où figurent Leonardo Dicaprio **et** Tom Hanks
 - `>db.films.find({"actors":{"$all":["Leonardo DiCaprio", "Tom Hanks"]}}, {"title":1,"year":1,"actors":1,_id:0}).pretty()`

```
{
  "title": "Catch Me If You Can"
  "year": 2002
  "actors": [
    "Leonardo DiCaprio",
    "Tom Hanks",
    "Christopher Walken",
    "Martin sheen"
  ]
}
```

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents
 - ✓ Opérateur de recherche : `$exists:true|false`
 - Vérifie si les documents possèdent ou non une telle clé (champs)
 - Les valeurs de `$exists` sont true (1) le champ existe pour ce document ou false (0) le champ n'existe pas pour ce document
 - Exemple : on désire afficher les films dont le champ « type » est précisé
 - `>db.films.find({"type":$exists:1})`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents
 - ✓ Le shell MongoDB s'appuie sur un interpréteur JavaScript:
 - Dans le shell, mongo affiche les 20 premiers, il faut entrer la commande **it** pour afficher la suite des résultats
 - Le résultat d'une requête renvoie un objet JavaScript. soit la base test :
 - `> use test`
 - `> db.expendables.insert( { name: 'Sylvester' } );`
 - `> db.expendables.insert( { name: 'Jean-Claude' } );`
 - `> db.expendables.insert( { name: 'Chuck' } );`
 - On peut déclarer une variable pour récupérer le résultat d'une requête
 - `>var single = db.expendables.findOne();`
 - `>single.name;`

Sylvester

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Recherche de documents
 - ✓ Le shell MongoDB s'appuie sur un interpréteur JavaScript:

- On peut utiliser la méthode `forEach()` :

- ```
>db.expendables.find().forEach(function(single){
 print(single.name);
});
```

Sylvester  
Jean-Claude  
Chuck

- On peut utiliser la boucle `while` pour parcourir un objet « `cursor` »

- ```
>var c = db.expendables.find()  
>while(c.hasNext()) print(c.next())
```

Sylvester
Jean-Claude
Chuck

Base de données NoSQL - MongoDB

■ Travaux Pratiques N°1

- Voir fiche de TP

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Mise à jour de documents
 - ✓ Méthodes : `update()` et `save()`
 - Méthode `update()` met à jour les valeurs dans le document existant
 - Syntaxe : `db.nom_collection.update(critère, maj_données, multi)`
 - Expl : `>db.testdata.update({x:4},{ $set:{x:11}})`
`WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })`
 - Qd **clé** dans `maj_données` ne figure pas dans le document, la donnée (clé:valeur) est **ajoutée**:
expl: `>db.testdata.update({x:4},{ $set:{y:11}})`
la donnée `y:11` est ajoutée au document de `x:4`
 - Par défaut 1 seul document est mise à jour, pour mettre plusieurs documents à jour, mettre le paramètre `multi` à `true`
`>db.testdata.update({x:4},{ $set:{y:11}}, {multi:true})`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Mise à jour de documents
 - Méthode `save()` remplace document $\exists$ tant par celui passé en paramètre
 - Syntaxe :
`db.nom_collection.save({_id:ObjectId(),nouvelle_donnée})`
 - Expl : `>db.testdata.save({"_id" : ObjectId("62af84157578f5eccebaf16e"), x:21, y:3, z:43})`
`WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })`
 - ✓ Méthodes : `findOneAndUpdate()`, `updateOne()` et `updateMany()`
 - La méthode `update()` est dépréciée, plutôt utiliser les méthodes précitées
 - Méthode `findOneAndUpdate(critère, maj_données)` met à jour les valeurs du document existant
 - Expl : `>db.testdata.findOneAndUpdate({x:21},{ $set:{y:12,z:20}})`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Mise à jour de documents
 - ✓ Méthodes : `findOneAndUpdate()`, `updateOne()` et `updateMany()`
 - Méthode `findOneAndUpdate(critère, maj_données)`
Expl : `>db.testdata.findOneAndUpdate({x:21},{ $set:{y:12,z:20}})`

```
{
  "_id" : ObjectId("62af84157578f5ecceba16e"),
  "x" : 21,
  "y" : 3,
  "z" : 43
}
```
 - Méthode `updateOne(critère, maj_données)` met à jour un seul document correspondant au critère donné
 - Expl : `>db.testdata.updateOne({x:5},{ $set:{x:14,a:3}})`

```
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1
}
```

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Mise à jour de documents
 - ✓ Méthodes : `findOneAndUpdate()`, `updateOne()` et `updateMany()`
 - Méthode `updateMany(critère, maj_données)` met à jour tous les documents correspondant au critère donné
 - Expl : `>db.testdata.updateMany({x:{$gt:5}},{$set:{x:0}})`
`{ "acknowledged" : true, "matchedCount" : 7, "modifiedCount" : 7 }`
 - ✓ La méthode : `replaceOne(critère, maj_données)`
 - Remplace un seul document
 - Expl : `>db.testdata.remove({x:0},1)`
`WriteResult({ "nRemoved" : 1 })`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Opérateurs avancés de mise à jour de documents
 - ✓ opérateur : **\$unset**
 - Supprime une clé (champ) dans le document
 - Expl : `>db.testdata.update({},{$unset: {"type":1}})`
supprime le champ type pour les documents qui en possèdent
 - ✓ opérateur : **\$push**
 - Ajoute une valeur à un tableau
 - Expl: `>db.people.update({name:'Tom'},{$push:{surnom:'Tommy'}})`
ajout de Tommy au tableau de surnom de Tom
 - L'opérateur **\$addToSet** évite les doublons
 - ✓ Opérateur : **\$pull**
 - Supprime un élément spécifique du tableau
 - Expl: `>db.people.update({name:'Tom'},{$pull:{surnom:'Tommy'}})`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Opérateurs avancés de mise à jour de documents
 - ✓ opérateur : **\$inc**
 - Ajoute une valeur à un champ numérique
 - Expl : `>db.cinema.update({nom:'The Godfather'},{$inc: {"annee":2}})`
incrément de 2 l'année du film « The Godfather »
 - ✓ Opérateur : **\$pop**
 - Supprime la 1^{ère} ou la dernière valeur du tableau
 - Soit le schéma : `{name:'Tom', age:28, marks:[50,60,70]}`
 - `>db.people.update({name:Tom}, {$pop: {marks:-1}})`
supprime le 1^{er} élément du tableau c-à-d 50
 - `>db.people.update({name:Tom}, {$pop: {marks:1}})`
supprime le dernier élément du tableau c-à-d 70

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Mise à jour de documents incorporés, opérateur $\$$
 - Soit le schéma : `{name:'Tom', age:28, marks:[50,60,70]}`
 - Mettre à jour les marques de Tom à 55 où elles étaient à 50
 - `>db.people.update({name:'Tom', marks:50},{ $set:{"marks.$":55}})`
`WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })`
- ✓ Tableau contenant documents
 - Soit le schéma : `{name:'Tom', age:28, marks: [{"subject":"English", "marks":90}, {"subject": "Maths", "marks": 100}, {"subject":"Computes", "marks":20}]}`
 - Mettre à jour les marques anglaises de Tom à 85
 - `>db.people.update({name:'Tom', "marks.subject": "English"}, { $set:{"marks.$.marks":85}})`
`WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Mise à jour de documents incorporés, opérateur $\$$
 - ✓ Pour le schéma :
 - `{"_id":1, "grades" : [80, 85, 90]}`
`{"_id":2, "grades" : [88, 90,92]}`
`{"_id":3, "grades" : [85, 100,90]}`
 - Mettre à jour 80 à 82 dans le tableau grades du 1^{er} document
 - `>db.students.update({_id:1, grades:80},{ $set: {"grades.$":82}})`
`WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })`
 - `> db.students.find().pretty()`
`{ "_id" : 1, "grades" : [ 82, 85, 90 ] }`
`{ "_id" : 2, "grades" : [ 88, 90, 92 ] }`
`{ "_id" : 3, "grades" : [ 85, 100, 90 ] }`

Base de données NoSQL - MongoDB

■ Utilisation de MongoDB

- Suppression de documents
 - ✓ Méthode : `remove(critère, [juste_un])`
 - Supprime les documents vérifiant le critère ou 1 seul quand le paramètre `juste_un` est défini à `true` ou `1`
 - Expl : `>db.testdata.remove({x:0},1)`
`WriteResult({ "nRemoved" : 1 })`
 - La méthode `remove()` est dépréciée, utiliser les méthodes ci-après
 - ✓ Méthode : `deleteOne(critère)`
 - Supprime le 1^{er} document vérifiant le critère
 - ✓ Méthode : `deleteMany(critère)`
 - Supprime tous les documents vérifiant le critère

Base de données NoSQL - MongoDB

■ Travaux Pratiques N°2

- Voir fiche de TP

Base de données NoSQL - MongoDB

■ Agrégation dans MongoDB

- But :
 - ✓ Traiter les enregistrements de données et renvoyer les résultats calculés
 - ✓ L'agrégation s'effectue souvent à travers un pipeline, un tableau d'opérations pouvant contenir les éléments suivants:
 - `$project` : redéfinition des documents (si nécessaire)
 - `$match` : restriction sur les documents à utiliser
 - `$group` : regroupements et calculs à effectuer
 - `$sort` : tri sur les agrégats
 - `$unwind` : découpage de tableaux

Base de données NoSQL - MongoDB

■ Agrégation dans MongoDB

- Agrégation de type pipeline
 - ✓ Syntaxe : `db.collection.aggregate(pipeline, options)`
 - pipeline : array ou tableau (une séquence d'opérations ou d'étapes d'agrégation de données)
 - options : facultatif (document disponible uniquement si le pipeline est présent sous forme de tableau)
 - ✓ Exemples simples : soient les données suivantes
 - > `db.transactions.insert({ cr_dr : "D", amount : 100, fee : 2 });`
 - > `db.transactions.insert({ cr_dr : "C", amount : 100, fee : 2 });`
 - > `db.transactions.insert({ cr_dr : "C", amount : 10, fee : 2 });`
 - > `db.transactions.insert({ cr_dr : "D", amount : 100, fee : 4 });`
 - > `db.transactions.insert({ cr_dr : "D", amount : 10, fee : 2 });`
 - > `db.transactions.insert({ cr_dr : "C", amount : 10, fee : 4 });`
 - > `db.transactions.insert({ cr_dr : "D", amount : 100, fee : 2 });`

Base de données NoSQL - MongoDB

■ Agrégation dans MongoDB

- Agrégation de type pipeline

- ✓ Exemples simples :

- On désire afficher le nombre d'opérations de débit. Une façon simple de le faire est :

- `>db.transactions.count({cr_dr : "D"});` ou

- `>db.transactions.find({cr_dr : "D"}).count();`

- Mais si on ne connaît pas les valeurs possibles de cr_dr, alors utiliser une agrégation s'avère utile (opérateur `$group`).

- `>db.transactions.aggregate([{$group : {
 _id: '$cr_dr', //grouper par type de transaction
 //ajouter 1 pour chaque document compté de ce type
 count : {$sum : 1}
}}]);`

Base de données NoSQL - MongoDB

■ Agrégation dans MongoDB

- Agrégation de type pipeline
 - ✓ Exemples simples :
 - Le résultat est :
 - { "_id" : "C", "count" : 3 }
 - { "_id" : "D", "count" : 4 }
 - Noter bien dans la requête le \$ placé avant le nom du champ (item) sur lequel s'effectue le regroupement. La syntaxe est toujours : « `_id:$nom_champ` »
 - après le champ de regroupement, on applique une fonction de calcul dont le nom de champ est choisi arbitrairement. Ici, on a choisi « count » et la fonction de calcul `$sum` avec une valeur de 1 pour spécifier que l'on compte

Base de données NoSQL - MongoDB

■ Agrégation dans MongoDB

- Agrégation de type pipeline

- ✓ Exemples simples :

- On désire sommer les valeurs du champ « amount »

- ```
> db.transactions.aggregate([
 $group : {_id : '$cr_dr', //définit le regroupement
 count : {$sum : 1}, //compte le nombre
 totalAmount : {$sum : '$amount'} //somme the amount
}]);
```

- Le résultat est :

- ```
{ "_id" : "C", "count" : 3, "totalAmount" : 120 }  
{ "_id" : "D", "count" : 4, "totalAmount" : 310 }
```

Base de données NoSQL - MongoDB

■ Agrégation dans MongoDB

- Agrégation de type pipeline

- ✓ Exemples simples :

- On peut calculer sur +sieurs champs: « amount » et « fee »

- ```
> db.transactions.aggregate([
 $group : {_id : '$cr_dr',
 count : {$sum : 1},
 totalAmount : {$sum : {$sum : ['$amount','$fee']}}
]]);
```

- Le résultat est :

- ```
{ "_id" : "C", "count" : 3, "totalAmount" : 128 }  
{ "_id" : "D", "count" : 4, "totalAmount" : 320 }
```

ici, on a fait la somme des amount et fee

Base de données NoSQL - MongoDB

■ Agrégation dans MongoDB

- Agrégation de type pipeline

- ✓ Exemples simples :

- On veut calculer la moyenne des transactions de débit et crédit

- ```
> db.transactions.aggregate([
 $group : {
 _id : '$cr_dr',
 count : {$sum : 1},
 totalAmount : {$sum : { $sum : ['$amount', '$fee']}},
 averageAmount : {$avg : { $sum : ['$amount', '$fee']}}
]})
```

- Le résultat est :

- ```
{ "_id" : "C", "count" : 3, "totalAmount" : 128, "averageAmount" : 42.66666666 }
```
 - ```
{ "_id" : "D", "count" : 4, "totalAmount" : 320, "averageAmount" : 80 }
```

# Base de données NoSQL - MongoDB

## ■ Agrégation dans MongoDB

- Agrégation de type pipeline

- ✓ Opérateur : `$match`

- Il permet de simuler les clauses SQL « `where` » et « `having` »

- Soient les données suivantes :

```
db.employees.insert({"name":"Adma","dept":"Admin","languages":["german","french","english","hindi"],
"age":40, "totalExp":10});
db.employees.insert({"name":"Anna","dept":"Admin","languages":["english","hindi"],"age":35,
"totalExp":11});
db.employees.insert({"name":"Bob","dept":"Facilities","languages":["english","hindi"],"age":36,
"totalExp":14});
db.employees.insert({"name":"Cathy","dept":"Facilities","languages":["hindi"],"age":31,
"totalExp":4});
db.employees.insert({"name":"Mike","dept":"HR","languages":["english", "hindi",
"spanish"],"age":26, "totalExp":3});
db.employees.insert({"name":"Jenny","dept":"HR","languages":["english", "hindi",
"spanish"],"age":25, "totalExp":3});
```

# Base de données NoSQL - MongoDB

## ■ Agrégation dans MongoDB

- Agrégation de type pipeline

- ✓ Opérateur : **\$match**

- On veut les documents du département « Admin »

- `>db.employees.aggregate([{$match: {dept:'Admin'}}])`

- Le résultat est :

```
{ "_id" : ObjectId("6380f00bf17e7aed2d1b0c68"), "name" : "Adma", "dept" : "Admin", "languages" : [
"german", "french", "english", "hindi"], "age" : 40, "totalExp" : 10 }
{ "_id" : ObjectId("6380f00bf17e7aed2d1b0c69"), "name" : "Anna", "dept" : "Admin", "languages" : [
"english", "hindi"], "age" : 35, "totalExp" : 11 }
```

- ✓ Opérateur : **\$project**

- Permet de spécifier les champs à afficher

- `>db.employees.aggregate([{$match:{dept:'Admin'}},
{$project: {_id:0, name:1, dept:1}}])`

# Base de données NoSQL - MongoDB

## ■ Agrégation dans MongoDB

- Agrégation de type pipeline

- ✓ Opérateur : `$project`

- Le résultat est :

- `{ "name" : "Adma", "dept" : "Admin" }`
- `{ "name" : "Anna", "dept" : "Admin" }`

- ✓ Opérateurs : `$match` et `$group` ensemble

- Afficher tous les départements où l'âge moyen des employés est supérieur ou égal à 35

- ```
>db.employees.aggregate([{$group: {_id:$dept,
                                moyenne_age:{$avg:$age}}},
                        {$match:{moyenne_age:{$gte:35}}} ])
```

- Le résultat est :

- `{ "_id" : "Admin", "moyenne_age" : 37.5 }`

Base de données NoSQL - MongoDB

■ Agrégation dans MongoDB

- Agrégation de type pipeline
 - ✓ Découpage de tableau, opérateur : `$unwind`
 - Soit à calculer le nombre de personnes par langue, une 1^{ère} idée est d'écrire :
 - `>db.employees.aggregate([ {$group: {_id:"$languages", nb:{$sum:1}}}] )`
 - Le résultat donne ceci :
 - `{ "_id" : [ "english", "hindi" ], "nb" : 2 }`
 - `{ "_id" : [ "german", "french", "english", "hindi" ], "nb" : 1 }`
 - `{ "_id" : [ "english", "hindi", "spanish" ], "nb" : 2 }`
 - `{ "_id" : [ "hindi" ], "nb" : 1 }`
 - Ce qui n'est pas correct, donc l'utilisation de `$unwind`

Base de données NoSQL - MongoDB

■ Agrégation dans MongoDB

- Agrégation de type pipeline
 - ✓ Découpage de tableau, opérateur : `$unwind`
 - La requête devient :
 - `>db.employees.aggregate([ {$unwind: "$languages"}, {$group: {_id:"$languages", nb:{$sum:1}}} ])`
 - Le résultat donne ceci :
 - `{ "_id" : "german", "nb" : 1 }`
 - `{ "_id" : "french", "nb" : 1 }`
 - `{ "_id" : "english", "nb" : 5 }`
 - `{ "_id" : "hindi", "nb" : 6 }`
 - `{ "_id" : "spanish", "nb" : 2 }`
 - Ce qui est convenable

Base de données NoSQL - MongoDB

■ Sauvegarde et restauration

- Importation de documents
 - ✓ L'on peut insérer des documents à partir d'un fichier json /csv
 - `mongoimport --db labase --collection "collection" --drop --type json --host "leserveur:port" --file "chemin_du_fichier"`
 - `--db` : nom de la base de données où importer les données
 - `--collection` : nom de la collection où les données seront stockées
 - `--drop` : supprime d'abord la collection avant d'importer
 - `--type` : précise le type de document. JSON par défaut
 - `--host` : hôte et port mongodb recevant les données
 - `--file` : chemin où se trouve le fichier json
 - Pour utiliser mongoimport il faut télécharger mongo tools command lines et les configurer si cela n'est pas déjà installé lors de l'installation

Base de données NoSQL - MongoDB

■ Sauvegarde et restauration

- Importation de documents

- ✓ Exemple :

- Soit à importer, les données au format json du fichier cinema.json situé dans le dossier « D:\Donnees_TP », dans la collection films de la base de données mabase. Lancer à partir du DOS, la commande suivante:
 - `C:\...>mongoimport --db mabase --collection "films" --drop --type json --file "D:/donnees_tp/cinema.json"`
 - Les données seront enregistrées dans la collection désignée. Ici, le serveur par défaut « localhost:27017 » a été utilisé, d'où l'omission de l'option « --host "serveur:port" »

Base de données NoSQL - MongoDB

■ Sauvegarde et restauration

- Exportation de documents
 - ✓ Pour exporter des documents dans un fichier json /csv
 - `mongoexport --db=labase --collection=collect --host "leserveur:port" --out "fichier_de_sortie" ou`
 - `Mongoexport --uri="mongodb://serveur:port/base" --collection=collect --out="fichier_de_sortie"`
 - `--db` : nom de la base de données où importer les données
 - `--collection` : nom de la collection où les données seront stockées
 - `--type` : précise le type de document. JSON par défaut
 - `--host` : hôte et port mongodb contenant les données
 - `--out` : le fichier qui recevra les documents
 - `--uri` : l'adresse de la base de données contenant les données

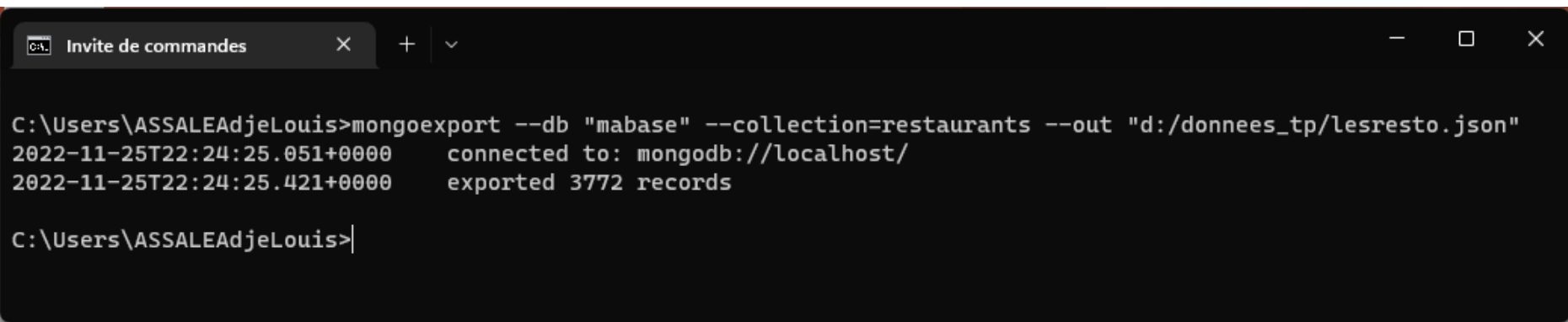
Base de données NoSQL - MongoDB

■ Sauvegarde et restauration

- Exportation de documents

✓ Exemple :

- Soit à exporter les documents de la collection restaurants de la base de données mabase vers le fichier lesRestaurants du dossier d:\donnees_tp
- `mongoexport --db=mabase --collection=restaurants --out "d:/données_tp/lessestos.json"`



```
Invite de commandes
C:\Users\ASSALEAdjéLouis>mongoexport --db "mabase" --collection=restaurants --out "d:/donnees_tp/lesresto.json"
2022-11-25T22:24:25.051+0000    connected to: mongod://localhost/
2022-11-25T22:24:25.421+0000    exported 3772 records
C:\Users\ASSALEAdjéLouis>
```

Base de données NoSQL - MongoDB

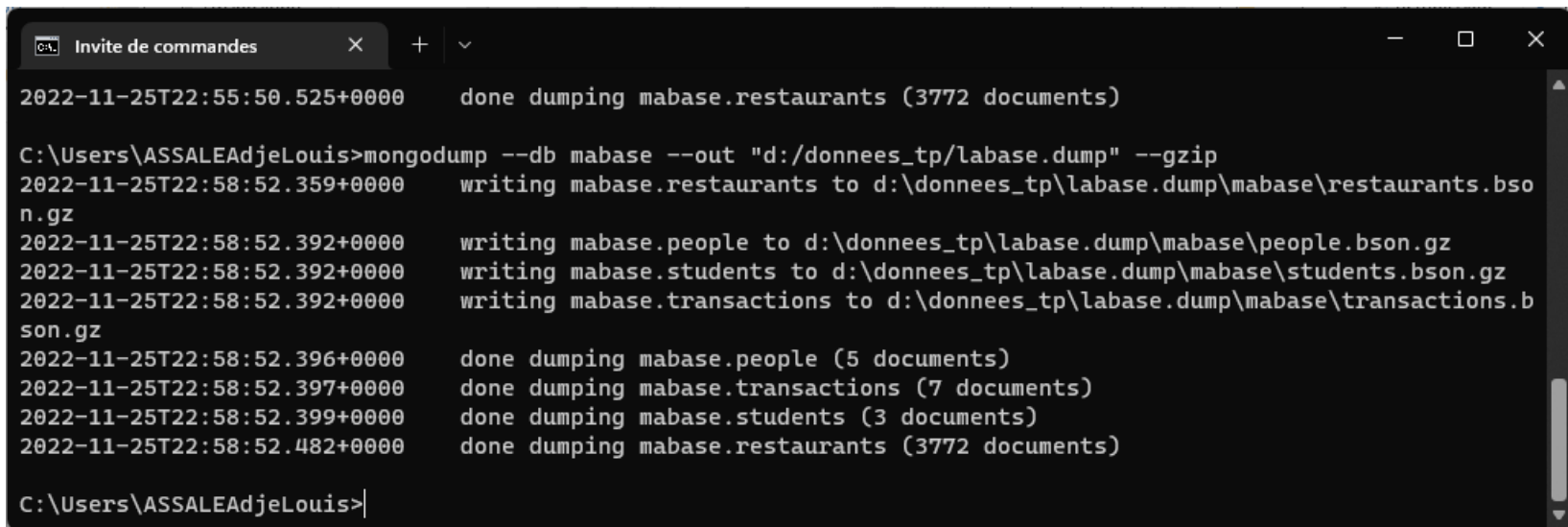
■ Sauvegarde et restauration

- Sauvegarde de bases de données
 - ✓ L'on peut sauvegarder toutes les collections d'une base de données
 - `mongodump --db=labase --host "leserveur:port" --out "fichier_de_sortie" --gzip --username user --password pass`
 - `--db` : nom de la base de données à sauvegarder
 - `--host` : hôte et port mongodb contenant les collections
 - `--out` : le fichier qui recevra la sauvgarde
 - `--gzip` : la sauvegarde sera compressée
 - `--username` : le nom utilisateur
 - `--password` : son mot de passe

Base de données NoSQL - MongoDB

■ Sauvegarde et restauration

- Sauvegarde de bases de données
 - ✓ Exemple :
 - Soit à sauvegarder la base de données « mabase »



```
Invite de commandes
2022-11-25T22:55:50.525+0000 done dumping mabase.restaurants (3772 documents)

C:\Users\ASSALEAdjéLouis>mongodump --db mabase --out "d:/donnees_tp/labase.dump" --gzip
2022-11-25T22:58:52.359+0000 writing mabase.restaurants to d:\donnees_tp\labase.dump\mabase\restaurants.bson.gz
2022-11-25T22:58:52.392+0000 writing mabase.people to d:\donnees_tp\labase.dump\mabase\people.bson.gz
2022-11-25T22:58:52.392+0000 writing mabase.students to d:\donnees_tp\labase.dump\mabase\students.bson.gz
2022-11-25T22:58:52.392+0000 writing mabase.transactions to d:\donnees_tp\labase.dump\mabase\transactions.bson.gz
2022-11-25T22:58:52.396+0000 done dumping mabase.people (5 documents)
2022-11-25T22:58:52.397+0000 done dumping mabase.transactions (7 documents)
2022-11-25T22:58:52.399+0000 done dumping mabase.students (3 documents)
2022-11-25T22:58:52.482+0000 done dumping mabase.restaurants (3772 documents)

C:\Users\ASSALEAdjéLouis>
```

Base de données NoSQL - MongoDB

■ Sauvegarde et restauration

- Restauration de bases de données
 - ✓ L'on peut restaurer une base de données sauvegardée
 - `mongorestore --host "leserveur:port" "fichier_de_sortie" --gzip --username user --password pass --drop`
 - `--host` : hôte et port mongodb contenant les collections
 - `--drop` : efface la base avant restauration
 - `--gzip` : la sauvegarde sera compressée
 - `--username` : le nom utilisateur
 - `--password` : son mot de passe

Base de données NoSQL - MongoDB

■ Sauvegarde et restauration

- Restauration de bases de données
 - ✓ Exemple : restauration de la base de données mabase

```
C:\Users\ASSALEAdjeLouis>mongorestore "d:\donnees_tp\labase.dump" --gzip --drop
2022-11-25T23:14:20.136+0000   preparing collections to restore from
2022-11-25T23:14:20.189+0000   reading metadata for mabase.people from d:\donnees_tp\labase.dump\mabase\people.metadata.json.gz
2022-11-25T23:14:20.189+0000   reading metadata for mabase.restaurants from d:\donnees_tp\labase.dump\mabase\restaurants.metadata.json.gz
2022-11-25T23:14:20.190+0000   reading metadata for mabase.students from d:\donnees_tp\labase.dump\mabase\students.metadata.json.gz
2022-11-25T23:14:20.191+0000   reading metadata for mabase.transactions from d:\donnees_tp\labase.dump\mabase\transactions.metadata.json.gz
2022-11-25T23:14:20.192+0000   dropping collection mabase.restaurants before restoring
2022-11-25T23:14:20.193+0000   dropping collection mabase.people before restoring
2022-11-25T23:14:20.193+0000   dropping collection mabase.transactions before restoring
2022-11-25T23:14:20.193+0000   dropping collection mabase.students before restoring
2022-11-25T23:14:20.252+0000   restoring mabase.restaurants from d:\donnees_tp\labase.dump\mabase\restaurants.bson.gz
2022-11-25T23:14:20.278+0000   restoring mabase.people from d:\donnees_tp\labase.dump\mabase\people.bson.gz
2022-11-25T23:14:20.309+0000   restoring mabase.transactions from d:\donnees_tp\labase.dump\mabase\transactions.bson.gz
2022-11-25T23:14:20.343+0000   restoring mabase.students from d:\donnees_tp\labase.dump\mabase\students.bson.gz
2022-11-25T23:14:20.375+0000   finished restoring mabase.people (5 documents, 0 failures)
2022-11-25T23:14:20.380+0000   finished restoring mabase.transactions (7 documents, 0 failures)
2022-11-25T23:14:20.384+0000   finished restoring mabase.students (3 documents, 0 failures)
2022-11-25T23:14:20.440+0000   finished restoring mabase.restaurants (3772 documents, 0 failures)
2022-11-25T23:14:20.440+0000   no indexes to restore for collection mabase.people
2022-11-25T23:14:20.441+0000   no indexes to restore for collection mabase.restaurants
2022-11-25T23:14:20.441+0000   no indexes to restore for collection mabase.students
2022-11-25T23:14:20.441+0000   no indexes to restore for collection mabase.transactions
2022-11-25T23:14:20.442+0000   3787 document(s) restored successfully. 0 document(s) failed to restore.

C:\Users\ASSALEAdjeLouis|
```

Base de données NoSQL - MongoDB

■ Travaux Pratiques N°3

- Voir fiche de TP